

SYNOPSIS REPORT

Title of the Project:

ATM Bank Management System

Submitted to: Manan Sir

Name: Arjun Shashikant Patil

Course / Roll No: 09

Department: BTech AI & DS

College / University: D. Y. Patil University, Ambi

Table of Contents

S. No	Content
1	Introduction
2	Methodology
2.1	Tools Used
2.2	Functions Used
2.3	Classes Used
2.4	Methods Implemented
2.5	System Flowchart
2.6	Architecture Diagram
3	Future Scope
4	Conclusion

1. Introduction

This Python project implements a Console-Based ATM Bank Management System that simulates real-world banking operations. The system allows users to create accounts, login, perform transactions (deposit, withdraw), check balances, view mini-statements, and change PINs. It emphasizes practical programming concepts such as file handling for persistent data storage, input validation, modular function design, and text-to-speech integration for enhanced accessibility. The project demonstrates structured development using Python's core capabilities without external dependencies (except pyttsx3 for TTS). The system uses a text file (data.txt) as a lightweight database to store account information, making it portable and easy to understand for educational purposes.

2. Methodology

2.1 Tools Used

- Python 3.14 (Core programming language)
- VS Code (Development environment)
- Libraries:
 - random – For generating random suggested PINs
 - os – For file existence checks
 - pyttsx3 – For text-to-speech functionality
- Database: Text file (data.txt) for persistent storage
- Platform: Windows

2.2 Functions Used

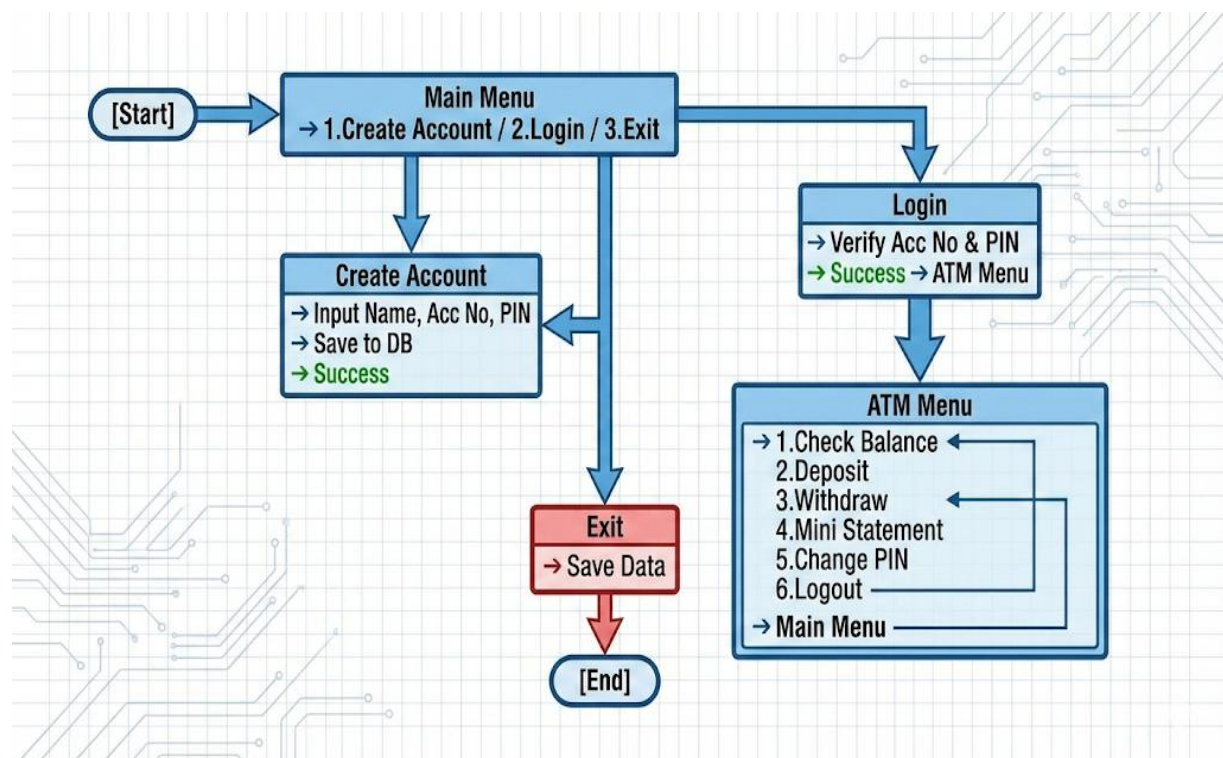
Function	Purpose
loaddb()	Loads account data from text file into memory
savedb()	Saves in-memory account data to text file
createaccount()	Creates a new bank account with validation
login()	Authenticates user using account number and PIN
checkbalance()	Displays current account balance
ministatement()	Shows last 5 transactions
deposit()	Adds money to account and updates transaction log
withdraw()	Deducts money with balance validation
changePIN()	Updates account PIN after verification
atmmenu()	Post-login menu for banking operations
main()	Main program loop and entry point

2.3 Methods Implemented

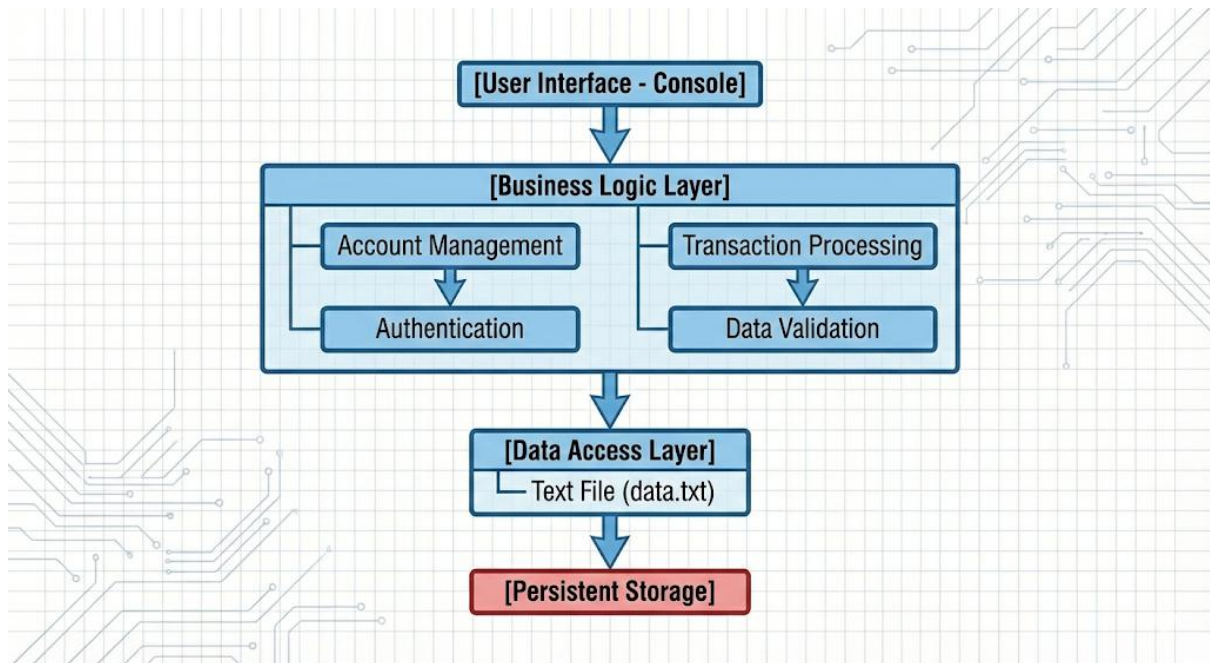
Since the project is function-based, the following key operations are implemented as functions:

- Validation Methods: Input validation for name (alphabets only), account number (10 digits), PIN (4 digits), and amount (positive integers)
- File Handling Methods: `loaddb()` and `savedb()` for persistent storage using text file
- Transaction Methods: `deposit()`, `withdraw()`, `checkbalance()` with balance updates
- Security Methods: `login()` with PIN verification, `changepin()` with confirmation
- UI/UX Methods: Text-to-speech feedback via `pyttsx3`, formatted console output, error handling
- Data Management: Dictionary-based in-memory storage (`bankdb`) with transaction logging

2.4 System Flowchart



2.5 Architecture Diagram



3. Future Scope

The ATM Bank Management System possesses significant potential for expansion and enhancement across multiple dimensions. A primary advancement would involve replacing the current console interface with a modern graphical user interface using frameworks like Tkinter, PyQt, or Kivy to significantly improve user experience and accessibility. Concurrently, migrating from the existing text file storage to a robust database system such as SQLite, MySQL, or PostgreSQL would enhance data integrity, scalability, and management capabilities. To support multi-user environments and simulate real-world banking networks, implementing a client-server architecture would be essential, allowing concurrent access from multiple terminals.

Security enhancements form another critical area for development, including the implementation of password hashing using bcrypt, encryption of sensitive data, and potentially introducing two-factor authentication for additional security layers. The system could be extended to mobile and web platforms by developing a REST API using frameworks like Flask or Django, enabling mobile banking applications and remote access. Adding an administrative module would provide comprehensive account management, transaction monitoring, and reporting functionalities for bank administrators.

4. Conclusion

The ATM Bank Management System successfully demonstrates core Python programming concepts including file I/O operations, dictionary manipulation, modular function design, input validation, and error handling. The project provides a functional banking simulation with persistent data storage using text files, text-to-speech feedback for accessibility, and a user-friendly console interface. Key achievements include:

- Robust data persistence through text file storage
- Comprehensive input validation and error handling
- Transaction history tracking with mini-statement feature
- Text-to-speech integration for enhanced user experience
- Modular design allowing easy maintenance and extension

While currently implemented using procedural programming, the system forms a strong foundation for future enhancement with Object-Oriented Programming principles, GUI development, database integration, and network capabilities. The project effectively balances simplicity with real-world banking requirements, making it an excellent educational tool for understanding software development lifecycle, Python application development, and basic banking system architecture. It serves as a practical demonstration of how programming concepts can be applied to solve real-world problems in the financial domain.